

Computer Architecture (Basics)

Computer Architecture defines the conceptual design & fundamental operational structure of a computer system.

- It acts as a **functional blueprint** for a **programmer**, specifying what the hardware can do.

Architecture vs organization vs Assembly language

• What the machine does (interface)	• How the machine implements it. (Hardware, circuit etc, buses etc).	• How the Humans instruct the interface. (software).
• High level functional blue-print	• low level physical micro architecture	• Human-readable representation of machine code.
• e.g: CPU supports a 32 bit add function using x1 & x2 register.	• e.g CPU uses a 5-stage pipeline & a carry-lookahead adder to complete instructions.	• add x1, x2,

Difference between RISC & CISC architecture:

Before in ~~an~~ COAL course we learned ~~more~~ MASM which is CISC architecture.

Now we will ~~be~~ learn RISC ~~an~~ architecture.

Main Difference:

RISC keeps instructions simple to optimise execution speed per cycle.

CISC provides complex instructions to minimize the number of lines of code needed per program.

Feature	RISC (e.g., RISC-V, ARM)	CISC (e.g., x86 / Intel & AMD)
Instruction Length	Fixed length (typically 32-bit). Easy to decode.	Variable length (1 to 15 bytes). Complex to decode.
Memory Access	Load-Store Architecture: Only dedicated instructions (LW, SW) touch memory. Operations happen in registers.	Memory-to-memory operations allowed directly (e.g., an ADD instruction can fetch from memory and store to memory).
Instruction Count vs. CPI	Higher instruction count per program, but low Clock Cycles Per Instruction (CPI) (mostly 1 cycle).	Lower instruction count per program, but high CPI (complex instructions take multiple cycles).
Hardware Complexity	Hardwired control logic; simple decoding; emphasizes software/compiler complexity.	Microcoded control units; complex hardware logic to break down heavy operations.
Register Set	Large general-purpose register file (e.g., 32 registers in RISC-V).	Smaller set of general-purpose and specialized registers.
Pipelining	Extremely straightforward due to fixed instruction sizes and uniform execution timing.	Difficult to pipeline directly (modern CISC chips convert CISC instructions into RISC-like micro-ops under the hood).

day/date

Instruction difference example:

In **CISC** e.g. x86 for ADD we had
So many ~~rules~~ different variants.

ADD ax, bx

ADD mem, ax

ADD ax, mem

- - - -

- - - .

- - - -

But in **RISC** we will only have ONE

ADD ax, bx, cx.

So it is a very simplified version.

A little bit about organization:

(How it is decided what ^{max} amount of RAM can be placed in a PC).

¶ We already know that there are different buses:

- 1) Data bus: transfers actual data
- 2) Address bus: ~~the~~ tells the address where data is stored.
- 3) Control bus

Now if the Address bus size is ~~8~~ 8-bit, then it can refer to max of 2^8 addresses. so only 256 bytes.

Thus if our Address bus is 8-bit we can have a maximum RAM of 256 bytes.

If our Address bus is ~~32~~ 32-bit, it can refer to 2^{32} addresses so 4096 MB or 4GB RAM.

so our RAM size depends upon our Address bus.

Secondly Data bus determines how many cycles would it take to bring the data.

e.g. an int is 32 bit but our data bus is 16-bit, it will take 2 cycles but if our data bus was 64-bit it can even transfer 2 ints in one go!

Measuring Performance

Consider we say one computer is better than other in performance. But what do we even mean by this statement?

Performance can be measured by different metrics. e.g. consider an airplane, its metrics can be speed, range & capacity.

But still consider one metric like ~~speed~~ of an airplane. Now consider 2 planes: one plane transports 2 passengers but in just 30 minutes, second plane transports 480 passengers but in 1.5 hrs. Now which is better?

Like with this same analogy applies to computers as well.

Performance depends heavily on the user.

A single user waiting for a program to finish cares about response time while a system administrator manager managing a server farm cares about through put time.

Response Time:

also called execution time.

The total time required for a computer to complete a task, including disk access, memory access, I/O activities, OS overhead CPU execution time etc.

Through put Time:

also called band width.

It is the number of tasks completed per unit time.

Performance:

Overall system performance for a program is defined as:

$$\text{Performance}_x = \frac{1}{\text{response time}_x}$$

OR.

$$\text{performance}_x = \frac{1}{\text{execution time}_x}$$

* relation is inversely proportional.

Comparing performance of 2 systems:

$$\frac{\text{performance}_x}{\text{performance}_y} = \frac{\text{execution time}_y}{\text{execution time}_x}$$

if n is > 1 , x has higher performance
 if n is < 1 , y has higher performance
 if n is $= 1$, same performance

Wall Clock time vs CPU Time:

~~CPU~~ **CPU Time** is actual time spent by the CPU actively calculating instructions for your specific program. ~~It includes time spent waiting for other apps~~

Wall clock time or Elapsed time: ~~is~~ ~~the~~ ~~total~~ ~~time~~ ~~to~~ ~~finish~~ ~~a~~ ~~task~~. It includes CPU time & time spent executing other OS tasks.

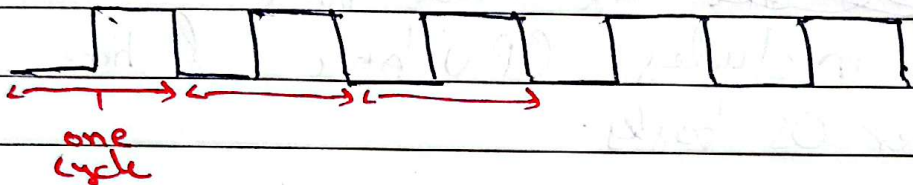
~~Elapsed time~~
CPU Time further

User CPU Time: The CPU time spent in a program itself.

System CPU Time: The CPU time spent in the OS performing task on behalf of program

Time & clock metrics:

All computers are driven by an internal clock that beats at a constant rate synchronizing every hardware event in the processor.



clock ~~is~~ beating

Now we will need to learn a few clock metrics because they are used in measuring performance.

Clock Cycle: This is the time for one clock period, which runs at constant rate. Also called *tick, clock tick, period*.

Clock Period: Length of each clock cycle.
~~also called clock cycle time.~~
 Unit: seconds (s)

Clock Rate: The number of clock cycles per second. (frequency).
unit = Hz (Hertz).

$$\text{clock cycle time} = \frac{1}{\text{clock rate (frequency)}}$$

Calculating execution time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time.}$$

$$\text{CPU execution time for a program} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate.}}$$

Example: Question: A program runs in 10 seconds on computer A which has 2 GHz clock. We are trying to design a computer B that will run program in 6 seconds. ~~The~~ You have seen that we can increase the clock rate but due to this design computer B requires 1-2 times as many clock cycles as A for the program. What will be the clock rate ~~rate~~ required for B?

computer A:

CPU time taken = 10 second.

Clock ~~rate~~ rate = 2 GHz.

of ~~CPU~~ clock cycles = ?

comp B:

CPU time = 6 second.

Clock rate = ?

of ~~CPU~~ clock cycles ~~is~~ = 1.2 x A's cycles.

~~Ans~~

$$\text{CPU time of A} = \frac{\text{\# of cycles of A}}{\text{clock rate of A.}}$$

$$\begin{aligned}\text{\# of cycles of A} &= 10 \times (2 \times 10^9) \\ &= 20 \times 10^9 = \del{20 \times 10^9}\end{aligned}$$

$$\begin{aligned}\text{\# of cycles of B} &= 1.2 \times 20 \times 10^9 \\ &= 24 \times 10^9.\end{aligned}$$

$$\text{CPU time of B} = \frac{\text{\# of cycles of B}}{\text{clock rate of B}}$$

$$6 = \frac{24 \times 10^9}{\text{clock rate of B}}$$

$$\begin{aligned}\text{clock rate of B} &= 4 \times 10^9 \\ &= 4 \text{ GHz}\end{aligned}$$

← answer

Instruction Performance:

The previous equations did not refer to the number of instructions needed for the program. But since a compiler generates instructions to execute, the execution time must depend upon number of instructions.

Example:

consider A & B do the same thing but with different instructions:

A	B
Add	MUL
Add	MUL
MUL	SUB.
INC	
SUB	
ADD	
# of instruction = 6	# of instruction = 3.

now consider all operations take 1 cycle
but MUL take 10 cycles.

so

A takes 15 cycles
B takes 21 cycles.

Average cycle Per instruction of A = $\frac{15}{6} = 2.5$ sec.

Average cycle Per instruction of B = $\frac{21}{3} = 7$ seconds.

(Average)

Clock cycles per instruction: is the average number of clock cycles each instruction takes to execute. it

• it is abbreviated as **CPI**

Number of clock cycles:

Using CPI we can also write number of clock cycles in its format.

$$\# \text{ of clock cycles} = \text{Total instructions} * \text{CPI}.$$

= Total instruction is also written as Instruction Count (IC).

Execution time,

Now by substituting # of clock cycles by instruction, (above formula) we can get execution time in terms of ~~clock~~ instructions.

$$\text{Execution time} = \frac{\text{Instruction Count} * \text{CPI}}{\text{clock rate}}.$$

$$\text{Execution time} = \text{Instruction Count} * \text{CPI} * \text{clock cycle time}.$$

CPU clock Cycles:

(# of clock cycles).

$$\text{CPU clock cycles} = IC \times CPI$$

$$\text{CPU clock cycles} = \frac{\text{CPU}^{\text{execution}} \text{ Time}}{\text{clock cycle time}}$$

$$\text{CPU clock cycles} = \text{CPU}^{\text{execution}} \text{ Time} \times \text{clock rate}$$

Variations by Instruction classes
(Multi class CPI)

When different types of instructions take different number of cycles, total cycles are calculated by summing the cycles for each class:

$$\text{CPU clock cycles} = \sum_{i=1}^n (\text{Instruction count}_i \times CPI_i)$$

Now as you can see CPI is not constant it varies off ~~the~~ by the instruction type class. Why?

actual number of cycles an instruction takes depends on dynamic execution content & can change on runtime.

Question: Comparing Code segments:

① A compiler designer is trying to decide between 2 code sequences for a particular computer.

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

The compiler designer is considering 2 code sequences that require following instruction count

Code sequence	Instruction count		
	A	B	C
1	2	1	2
2	4	1	1

Which code sequence executes most instructions? which will be faster.
What is CPI of each?

Answer

Comparing number of instructions:

Instruction in sequence 1 = $2+1+2=5$

Instruction in sequence 2 = $4+1+1=6$.

sequence 1 is better.

Comparing number of clock cycles:

CPU clock cycles in ^{sequence 1} A = $(2 \times 1) + (1 \times 2) + (2 \times 3)$
= 10 cycles.

CPU clock cycles in ^{sequence 2} B = $(4 \times 1) + (1 \times 2) + (1 \times 3)$
= 9 cycles.

so sequence 2 is faster.

Comparing CPI of each:

now to find this we simply find average CPI of each sequence:

$$\text{CPI of 1} = \frac{10}{(2+1+2)} = 2$$

$$\text{CPI of 2} = \frac{9}{4+1} = 1.5$$

so sequence 2 is faster.

Imp points:

CPI & CPI_i are different.

CPI is the average for the full program. so

$$\text{CPI} = \frac{\text{total cycles}}{\text{All Instruction count.}}$$

CPI_i is of a particular instruction class.

$$\text{CPI}_i = \frac{(\text{total cycles required by instruction class } i)}{\text{\# of instructions of } i.}$$

How to reduce CPU Time?

There are 3 ways we can reduce CPU time:

1) Reduce instruction Count:

- This depends on instruction set architecture + compiler technology + algorithm + language selection

2) Reduce CPI

- which depends on system organization (pipelined or not) & instruction set architecture.

3) Reduce clock cycle time (or increase rate)

- which depends on technology (VLSI/ ULSI) implementation (hard wired vs M-programmed) & instruction set architecture.

* Notice that reducing one measure can increase other. E.g. one can reduce instruction count by using complex instructions but this will likely lead to increase in clock cycles.

BHATTI

//date

MIPS as a performance Measure:

MIPS: Millions of instructions per second.

Faster Machine = larger MIPS

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{Execution time} \times 10^6} = \frac{\text{clock rate}}{\text{CPI} \times 10^6}$$

$$\text{Execution time} = \frac{\text{Instruction Count}}{\text{MIPS} \times 10^6} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

MIPS example:

Q: Two different computers are being tested on the same program for a 4 GHz machine with 3 different classes of instructions: class A, class B & class C which require 1, 2, 3 cycles respectively.

The instructions count by first computer is 5 billion class A, 1 billion class B & 1 billion class C.

The second computer produced - 10 billion class A, 1 billion class B & 1 billion class C.

~~Q: Q:~~

a) which computer produces higher MIPS.

b) which computer produces a better execution time

Solution.

	Class		
	A	B	C
CPI _i	1	2	3
Computer 1 count	5×10^9	1×10^9	1×10^9
Computer 2 count	10×10^9	1×10^9	1×10^9

Computer 1:

$$\begin{aligned}
 \text{Total CPI} &= \frac{\sum (\text{CPI}_i * \text{count}_i)}{\text{Total Instr. Count}} \quad \text{Total cycles.} \\
 &= \frac{(1 \times 5 + 2 \times 1 + 3 \times 1) \times 10^9}{(5 + 1 + 1) \times 10^9} \\
 &= 1.428.
 \end{aligned}$$

~~Total cycles~~

$$\begin{aligned}
 \text{Execution time} &= \frac{\text{Total cycles}}{\text{Clock rate}} \\
 &= \frac{(1 \times 5 + 2 \times 1 + 3 \times 1) \times 10^9}{4 \times 10^8} \\
 &= 2.5 \text{ seconds.}
 \end{aligned}$$

day/date

$$\begin{aligned} \text{MIPS} &= \frac{\text{Instruction Count}}{\text{Execution time} \times 10^6} \\ &= \frac{(5+1+1) \times 10^9}{2.5 \times 10^6} = 2800 \end{aligned}$$

Computer 2:

$$\begin{aligned} \text{Total CPI} &= \frac{(10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9}{(12) \times 10^9} \\ &= 1.25 \end{aligned}$$

$$\begin{aligned} \text{Execution time} &= \frac{(10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9}{4 \times 10^9} \\ &= 3.75 \text{ seconds.} \end{aligned}$$

$$\text{MIPS} = \frac{12 \times 10^9}{3.75 \times 10^6} = 3200.$$

so computer 1 was a faster program due to less CPU time taken but computer 2 had a higher MIPS rating.

BHATTI

Drawbacks of MIPS:

* Ignores the capability of instructions.
cannot use MIPS to compare computers with different instruction sets because instruction count will differ.

* MIPS varies between programs on same computer
a computer cannot have a single MIPS rating for all programs.

* Can move inversely with real performance.
A higher MIPS does not mean better performance like we saw in previous example.

THUS MIPS is called a meaning-
less indicator of processor speed.

Evaluating Performance gain.

Now we will see how we can evaluate 2 machines for performance gain.

In this a common mistake: It is not necessary that an improvement of one aspect of a computer to increase performance will lead to an increase by an amount of the size of improvement.

maybe we bring a big improvement but performance gain remains small.

§ Now we will understand speed up and Amdahl's Law to ~~understand~~ evaluate performance gain.

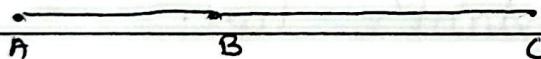
Speed Up:

Speed Up measures the relative performance gain of an enhanced system compared to an original (unenhanced) baseline system when executing the same task.

$$\text{Speed Up (S)} = \frac{\text{Performance (Enhanced)}}{\text{Performance (Old)}} = \frac{\text{Execution time (Old)}}{\text{Execution time (Enhanced)}}$$

Example:

Consider 2 trips:



from A to B, constant 20 seconds are taken.

but for B to C it is dynamic & we can change ^{dist} from 50, 20, 4, 1.7 & 0.3. compares all these & find speed up between B & C & over all speed up.

day/date

* we are considering B-C = 50 seconds as the baseline (old).

A-B mp	B-C mp	Total km	B-C speedup	Total speedup
20	50	70	1	1
20	20	40	2.5	1.75
20	4	24	12.5	2.9
20	1.7	21	29.4	3.3
20	0.3	20.3	166.66	3.4

Now as you can see B-C speedup is increasing by a huge rate but total speed up is slowing down because it depends on the A-B factor more now.

This is well explained by Amdahl's law.

Amdahl's Law (simpler explanation):

The performance improvement to be gained from using some faster mode of execution is limited by the **fraction** of the time the faster mode cannot be used.

* **lowest performer restricts all other**

Amdahl's Law calculates the overall speed up achieved when only a **portion** of a program or system is improved.

Execution time after improvement = $\frac{\text{Execution time affected by Improvement}}{\text{Amount of improvement}} + \text{execution time unaffected.}$

* This can be understood better by example on next page.

Example:

Q: Total time to execute a program is set to 1. The non-parallelizable part of program is 40 % ~~at out~~ out of total time of 1.

a) Consider we want to improve ~~non-parallelizable~~ ^{parallelizable} part by a factor of 2. Then new time = ?

$$\text{new time} = \text{unaltered time} + \left(\frac{\text{affected time}}{\text{change factor}} \right)$$

non-parallelizable time *parallelizable*

$$= 0.4 + \frac{(1-0.4)}{2}$$

$$\text{new time} = 0.7.$$

so our time reduced ~~by~~ to 0.7 from 1.

b) improve ~~the~~ parallelizable part by 5 & non parallelizable part by 2:

Now we are improving both part of programs by different factors :

new time = new time of parallelizable^{part} +
new time of non-parallelizable part

$$= \left(\frac{0.6}{5} \right) + \left(\frac{0.4}{2} \right)$$

$$= 0.32.$$

Imp:

To do these Amdahl's questions:

- basically divide the particular time into the provided parts.
- Then for each part check the improving factor (if not ~~is~~ being improved their time remain same)

- Then change the parts times accordingly to their improvement by:

$$\text{new part time} = \frac{\text{Particular Part time}}{\text{speed up factor}}$$

- add all parts times up.

$$\text{new time} = \text{Part 1 (new time)} + \text{Part 2 (new time)} \dots + \text{part n (new time)}$$

* a part's new time can be exactly old time if it wasn't improved.

Speed up example 2:

Q: A program 100 seconds on a computer with multiply operation responsible for 80 seconds of this time.

How much should we improve speed of multiplication if we want program to run 5 times as fast.

So we need to find the speedup factor for multiplication part.

$$\underbrace{\frac{100}{5}}_{\text{new time}} = \underbrace{100}_{\text{unaffected}} - \underbrace{80}_{\text{affected time}} + \frac{80}{n}$$

5 times fast

$$20 = 20 + \frac{80}{n}$$

$$0 = \frac{80}{n}$$

not possible. So we can't reach 5 times as fast because lowest performer restricts all others.

Amdahl's Law (Actual formula).

Before we used a very simpler version of the formula but basically it conveys the same meaning as the original formula we will discuss now.

$$ET_{\text{new}} = ET_{\text{old}} * \left[\left(1 - f_{\text{enhance}}\right) + \frac{f_{\text{enhanced}}}{S_{\text{enhanced}}} \right]$$

ET_{new} = execution time new

ET_{old} = execution time old.

f_{enhanced} = fraction of the total execution time that can be improved.

e.g

If a part that takes 10 seconds out of total 40 s execution time & can be improved.

$$f_{\text{enhance}} = \frac{10}{40} = 0.25$$

day/date

$S_{\text{enhanced}} = \text{Speed up factor applied for improvement} \cdot (\text{Amount of improvement})$

Total Speed up:

$$\text{Speedup}_{\text{overall}} = \frac{\text{Performance}_{\text{(new)}}}{\text{Performance}_{\text{(old)}}} = \frac{E_{\text{(old)}}}{E_{\text{(new)}}}$$

$$\text{ExTime}_{\text{new}} = \text{ExTime}_{\text{old}} \times \left[(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right] \rightarrow \text{Amdahl's law}$$

$$\text{Speedup}_{\text{overall}} = \frac{\text{ExTime}_{\text{old}}}{\text{ExTime}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

- speed up written using Amdahl's law.

Best you could ever hope to do:

$$\text{Speedup}_{\text{maximum}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}})}$$

Question:

Suppose we are considering enhancement of a processor on a server used for web. new cpu is 10 times faster. original cpu was busy with 40% of work & 60% I/O, what is overall speed up by new cpu.

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - 0.4) + \frac{0.4}{10}}$$

$$\boxed{= 1.56}$$

Amdahl's Corollary

- Make the common case faster

e.g. In addition overflow is not common so favor optimizing the case when no overflow occurs.

Objective:

- * Find frequent case
- * Determine possible feature improvement to make it fast

If an enhancement is only useful for a fraction of the task, we can't speed up the task by more than reciprocal of 1 minus that fraction

$$\text{Performance Improvement limit} \leq \frac{1}{1 - \text{fraction enhanced}}$$

Examples:

Q: See ~~the~~ ^a ~~designs~~ certain program/~~as~~ work.

~~Designs~~:

Frequency of FP instruction: 25%

Average CPI of FP instructions = 4.0

Average CPI of other instruction: 1.33

Frequency of FPSQR = 2%

✓
This is part of the FP instruction's 25%.

CPI of FPSQR = 20.

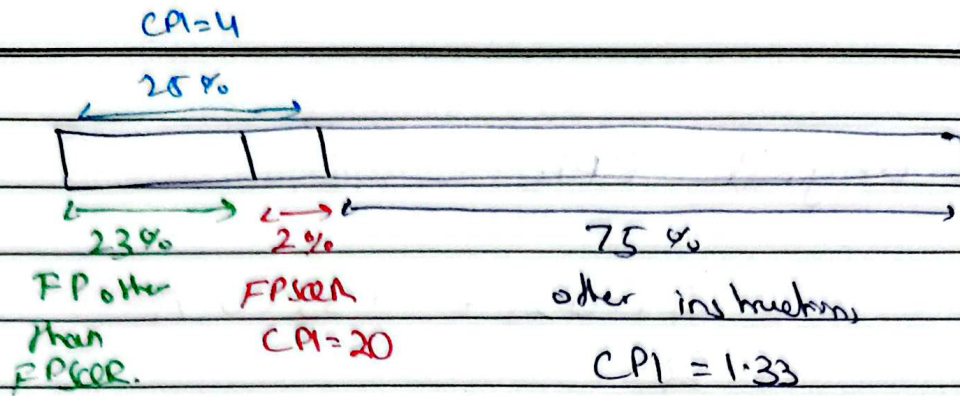
2 designs:

A: Reduce CPI of FPSQR from 20 to 2.

B: Reduce Avg CPI of FP instructions to 2.

Compare both which is better

y/date



Find CPI of FP other than FPSQR:

$$\text{Average CPI of FP} = \frac{2 \times 20 + \frac{23}{25} \times x}{25}$$

$$\frac{4 \times 25 - 40}{23} = x$$

$$x = 2.61$$

so Avg CPI of FP (except FPSQR) = 2.61.

Design 1 CPI overall:

$$= 2.61 \times 0.23 + 0.02 \times 2 + 1.33 \times 0.75$$
$$= 1.64$$

Design 2 CPI overall:

$$= 0.25 \times 2 + 0.75 \times 1.33$$
$$= 1.5$$

~~2.61~~ 2.61

day/date

original CPI:

$$= 4 \times 0.25 + 1.33 \times 0.75$$
$$= 2$$

$$\text{Speed up}_{\text{design 1}} = \frac{2}{1.64} = 1.22$$

$$\text{speed up}_{\text{design 2}} = \frac{2}{1.5} = 1.33$$

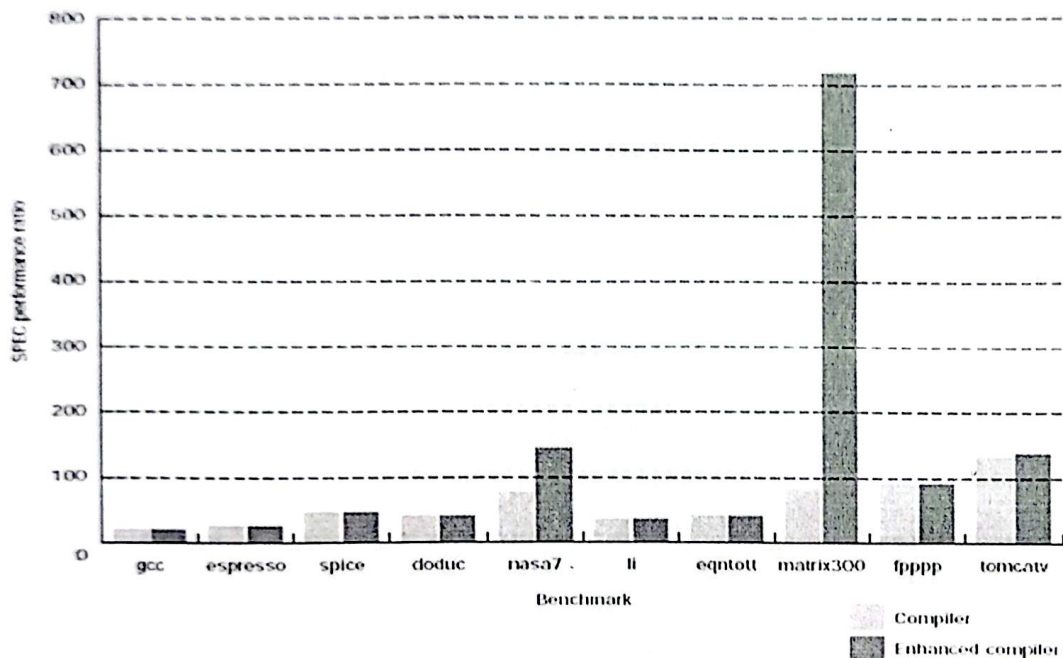
so design 2 is better.

Evaluating Performance

- How to measure Performance?
- How to report and summarize Performance?
- Workload (real applications)
vs.
- Benchmarks (programs specifically chosen to measure performance)

SPEC Benchmarks

(System Performance Evaluation Cooperative)
suite of benchmarks



Performance Report

- Key idea is Reproducibility
- Report consists of:
 - System description or machine configuration (CPU, FPU, # of CPU, cache and memory size, etc.)
 - Operating System and Compiler (OS type and rev, compiler rev, file system)
 - Workload & benchmarks
 - Choice of data input